

Bike-Sharing Analytics System

Τεχνολογίες Αποκεντρωμένων Δεδομένων

Εργασία εξαμήνου

Γεώργιος Μπαλλής 1040696

Μάριος Νεζερίτης 1080400

Χριστίνα Γιαννέλη 1090055

Λάμπης Γιαννέλης 1093341

Περιεχόμενα

Εισαγωγή.....	1
Spark Container.....	9
Εκπαίδευση μοντέλου παλινδρόμησης	15
Αποτελέσματα.....	18
Συμπεράσματα.....	19

Εισαγωγή

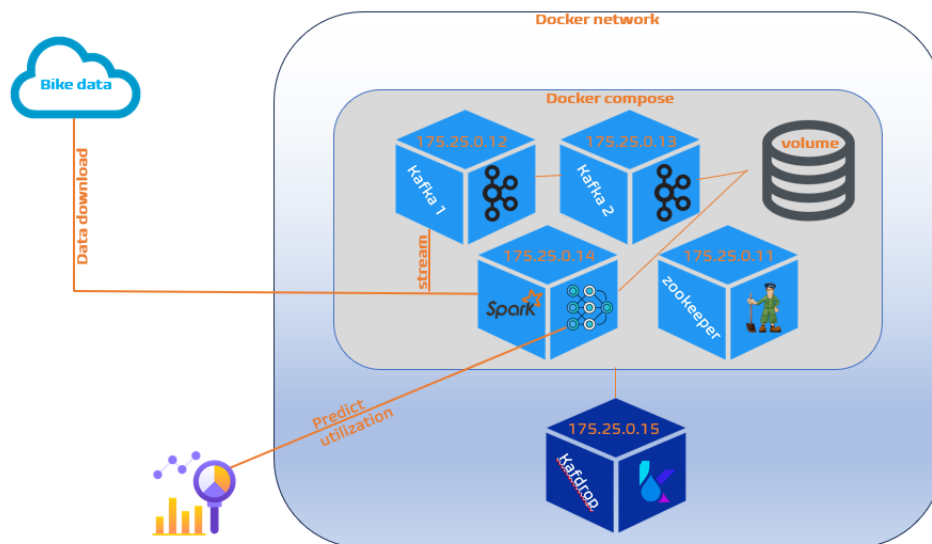
Το σύστημα που έχει υλοποιηθεί πρόκειται για μία αποτελεσματική και επεκτάσιμη προσέγγιση σχετικά με την ανάλυση της χρήσης ποδηλάτων. Η αρχιτεκτονική του που ενσωματώνει τεχνικές data streaming, data analysis αλλά και cloud τεχνολογίες, μέσω του Docker, αποτελεί ένα σύνολο διασυνδεδεμένων υπηρεσιών που εκτελούνται σε containers για να διασφαλιστεί μια επεκτάσιμη και ανεκτική σε σφάλματα υλοποίηση. Τα βασικά στοιχεία περιλαμβάνουν:

Kafka Cluster: Δύο Kafka brokers διαχειρίζονται τα δεδομένα ροής. Συντονίζονται από το Zookeeper, διασφαλίζοντας υψηλή διαθεσιμότητα και συγχρονισμό.

Spark Cluster: Το Spark επεξεργάζεται τα εισερχόμενα δεδομένα ροής, εκτελώντας βασικούς μετασχηματισμούς και τα ερωτήματα της ανάλυσης των δεδομένων. Χρησιμοποιεί επίσης ως η μηχανή για την εκτέλεση του Random Forrest.

Τέλος το σύστημα παρακολουθείται χρησιμοποιώντας το Kafdrop, μια διεπαφή ιστού που συνδέεται με το Kafka.

Το μοντέλο παλινδρόμησης προβλέπει τη χρήση των σταθμών ποδηλάτων για την επόμενη ώρα, χρησιμοποιώντας χαρακτηριστικά όπως η θερμοκρασία, η ταχύτητα ανέμου, η χωρητικότητα κάθε σταθμού και η ώρα της ημέρας καταγράφοντας το μέσο τετραγωνικό σφάλμα αλλά και το μέσο απόλυτο σφάλμα. Απεικονίζεται ένα high-level σχήμα της αρχιτεκτονικής



Όπως φαίνεται και από το σχήμα αρχικά στο container που εκτελείται το spark έχουν υλοποιηθεί τρεις κώδικες που λειτουργούν ως producers στέλνοντας τα δεδομένα στα 3 ξεχωριστά topics:

- *station_status* (https://gbfs.citibikenyc.com/gbfs/en/station_status.json)
- *station_information* (https://gbfs.citibikenyc.com/gbfs/en/station_information.json)
- *weather_data* (<https://api.open-meteo.com/v1/forecast>)

Ακολουθούν snippets από τα τρία scripts μαζί με τα αποτελέσματα τους από τα μηνύματα στο κάθε kafka topic

Station status

```
import logging
import json
import requests
import time
from kafka import KafkaProducer
from kafka.admin import KafkaAdminClient, NewTopic
```

```

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
BOOTSTRAP_SERVERS = ['172.25.0.13:9092'] # kafka ip:port
STATION_STATUS_TOPIC = "station_status" # kafka topic name

STATION_STATUS_URL = "https://gbfs.citibikenyc.com/gbfs/en/station_status.json" # api for station
status

admin_client = KafkaAdminClient(bootstrap_servers=BOOTSTRAP_SERVERS) # initialization of the
producers
producer = KafkaProducer(
    bootstrap_servers=BOOTSTRAP_SERVERS,
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

def create_topics(): # topic creation
    existing_topics = admin_client.list_topics()
    topics_to_create = []

    if STATION_STATUS_TOPIC not in existing_topics:
        topics_to_create.append(NewTopic(name=STATION_STATUS_TOPIC, num_partitions=1,
replication_factor=1))

    if topics_to_create:
        admin_client.create_topics(new_topics=topics_to_create, validate_only=False)
        logging.info("Created topics: {}".format([topic.name for topic in topics_to_create]))
    else:
        logging.info("Topics already exist.")

def fetch_station_status(): # api call
    try:
        response = requests.get(STATION_STATUS_URL)
        response.raise_for_status()
        return response.json()
    except requests.RequestException as e:
        logging.error("Error fetching data from {}: {}".format(STATION_STATUS_URL, e))
        return None

def send_station_status(): # data send to kafka topic
    station_status = fetch_station_status()
    if station_status:
        try:
            stations_data = station_status.get("data", {}).get("stations", [])
            for station in stations_data:
                producer.send(STATION_STATUS_TOPIC, value=station)
            producer.flush()

```

```

        logging.info("Sent station status data to Kafka.")
    except Exception as e:
        logging.error("Failed to send station status to Kafka: {}".format(e))

if __name__ == "__main__":
    logging.info("Starting Station Status Producer...")
    create_topics()

    try:
        while True:
            send_station_status()
            logging.info("Data sent successfully! Waiting for the next cycle...")
            time.sleep(100)
    except KeyboardInterrupt:
        logging.info("Shutting down producer...")
        producer.close()

```

Το αποτέλεσμα από την κονσόλα

```

2024-12-12 14:06:34,645 - INFO - Broker version identified as 2.2.0
2024-12-12 14:06:34,645 - INFO - Set configuration api_version=(2, 2, 0) to skip auto check_version requests on startup
2024-12-12 14:06:34,646 - INFO - Starting Station Status Producer...
2024-12-12 14:06:34,650 - INFO - Topics already exist.
2024-12-12 14:06:35,612 - INFO - <BrokerConnection node_id=1002 host=172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]: connecting to 172.25.0.13:9092 [('172.25.0.13', 9092) IPv4]
2024-12-12 14:06:35,613 - INFO - <BrokerConnection node_id=1002 host=172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]: Connection complete.
2024-12-12 14:06:35,613 - INFO - <BrokerConnection node_id=bootstrap-0 host=172.25.0.13:9092 <connected> [IPv4 ('172.25.0.13', 9092)]: Closing connection.
2024-12-12 14:06:36,023 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:36,023 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:06:38,810 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:38,811 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:06:41,625 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:41,626 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:06:44,777 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:44,778 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:06:47,715 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:47,716 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:06:50,693 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:50,694 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:06:53,478 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:53,479 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:06:56,267 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:56,268 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:06:59,041 - INFO - Sent station status data to Kafka.
2024-12-12 14:06:59,041 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:07:02,289 - INFO - Sent station status data to Kafka.
2024-12-12 14:07:02,289 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:07:05,052 - INFO - Sent station status data to Kafka.
2024-12-12 14:07:05,053 - INFO - Data sent successfully! Waiting for the next cycle...
^C2024-12-12 14:07:05,696 - INFO - Shutting down producer...

```

Τα μηνύματα που περνάνε στο container του αντίστοιχου kafka topic

```

PS C:\Users\lampis\Desktop\docker-kafka-spark-master> docker exec -it kafkal kafka-console-consumer.sh --bootstrap-server 172.25.0.13:9092 --topic station_status --from-beginning --max-messages 5
{"station_id": "0bd9bd58-42e6-4680-9d19-83943372221f", "num_ebikes_available": 0, "num_bikes_available": 0, "legacy_id": "4395", "num_bikes_disabled": 0, "is_renting": 0, "num_docks_disabled": 0, "eightd_has_available_keys": false, "last_reported": 1729514620, "is_installed": 0, "num_docks_available": 0, "is_returning": 0}
{"station_id": "816e58eb-dc4b-47dc-b773-154e2020cb0d", "num_ebikes_available": 0, "num_bikes_available": 0, "legacy_id": "4391", "num_bikes_disabled": 0, "is_renting": 0, "num_docks_disabled": 0, "eightd_has_available_keys": false, "last_reported": 1730827344, "is_installed": 0, "num_docks_available": 0, "is_returning": 0}
{"station_id": "566a6389-5c22-49ca-8c24-4d9eeal35170", "num_ebikes_available": 0, "num_bikes_available": 0, "legacy_id": "4393", "num_bikes_disabled": 0, "is_renting": 0, "num_docks_disabled": 3, "eightd_has_available_keys": false, "last_reported": 1731339673, "is_installed": 0, "num_docks_available": 0, "is_returning": 0}
{"station_id": "66dc7f02-0aca-11e7-82f6-3863bb44ef7c", "num_ebikes_available": 0, "num_bikes_available": 0, "legacy_id": "517", "num_bikes_disabled": 0, "is_renting": 0, "num_docks_disabled": 0, "eightd_has_available_keys": false, "last_reported": 1733268446, "is_installed": 1, "num_docks_available": 107, "is_returning": 0}
{"num_docks_available": 18, "num_ebikes_available": 1, "num_bikes_available": 3, "legacy_id": "1815968237323833702", "num_bikes_disabled": 0, "num_docks_disabled": 0, "is_renting": 1, "num_scooters_available": 0, "eightd_has_available_keys": false, "num_scooters_unavailable": 0, "last_reported": 1733845304, "is_installed": 1, "station_id": "1815968237323833702", "is_returning": 1}
Processed a total of 5 messages
PS C:\Users\lampis\Desktop\docker-kafka-spark-master>

```

Station information

```
import logging
import json
import requests
import time
from kafka import KafkaProducer
from kafka.admin import KafkaAdminClient, NewTopic

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
BOOTSTRAP_SERVERS = ['172.25.0.13:9092'] # kafka ip:port
STATION_INFO_TOPIC = "station_information" # kafka topic name

STATION_INFO_URL = "https://gbfs.citibikenyc.com/gbfs/en/station_information.json" # api for station
info

admin_client = KafkaAdminClient(bootstrap_servers=BOOTSTRAP_SERVERS) # initialization of the
producer
producer = KafkaProducer(
    bootstrap_servers=BOOTSTRAP_SERVERS,
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

def create_topics(): # topic creation
    existing_topics = admin_client.list_topics()
    topics_to_create = []

    if STATION_INFO_TOPIC not in existing_topics:
        topics_to_create.append(NewTopic(name=STATION_INFO_TOPIC, num_partitions=1,
replication_factor=1))

    if topics_to_create:
        admin_client.create_topics(new_topics=topics_to_create, validate_only=False)
        logging.info("Created topics: {}".format([topic.name for topic in topics_to_create]))
    else:
        logging.info("Topics already exist.")

def fetch_station_info(): # api call
    try:
        response = requests.get(STATION_INFO_URL)
        response.raise_for_status()
        return response.json()
    except requests.RequestException as e:
        logging.error("Error fetching data from {}: {}".format(STATION_INFO_URL, e))
        return None
```

```
def send_station_info(): # data send to kafka topic
    station_info = fetch_station_info()
    if station_info:
        try:
            stations_data = station_info.get("data", {}).get("stations", [])
            for station in stations_data:
                producer.send(STATION_INFO_TOPIC, value=station)
            producer.flush()
            logging.info("Sent station information data to Kafka.")
        except Exception as e:
            logging.error("Failed to send station information to Kafka: {}".format(e))

if __name__ == "__main__":
    logging.info("Starting Station Status Producer...")
    create_topics()

    try:
        while True:
            send_station_status()
            logging.info("Data sent successfully! Waiting for the next cycle...")
            time.sleep(100)
    except KeyboardInterrupt:
        logging.info("Shutting down producer...")
        producer.close()
```

Το αποτέλεσμα από την κονσόλα

```
2024-12-12 14:22:36,042 - INFO - <BrokerConnection node_id-bootstrap-0 host-172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]>: Connection complete.
2024-12-12 14:22:36,150 - INFO - Broker version identified as 2.2.0
2024-12-12 14:22:36,150 - INFO - Set configuration api_version=(2, 2, 0) to skip auto check_version requests on startup
2024-12-12 14:22:36,157 - INFO - Starting Station Status Producer...
2024-12-12 14:22:36,168 - INFO - Topics already exist.
2024-12-12 14:22:37,181 - INFO - <BrokerConnection node_id=1002 host-172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]>: connecting to 172.25.0.13:9092 [('172.25.0.13', 9092) IPv4]
2024-12-12 14:22:37,192 - INFO - <BrokerConnection node_id=1002 host-172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]>: Connection complete.
2024-12-12 14:22:37,196 - INFO - <BrokerConnection node_id-bootstrap-0 host-172.25.0.13:9092 <connected> [IPv4 ('172.25.0.13', 9092)]>: Closing connection.
2024-12-12 14:22:37,704 - INFO - Sent station information data to Kafka.
2024-12-12 14:22:37,706 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:22:40,764 - INFO - Sent station information data to Kafka.
2024-12-12 14:22:40,766 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:22:43,715 - INFO - Sent station information data to Kafka.
2024-12-12 14:22:43,717 - INFO - Data sent successfully! Waiting for the next cycle...
2024-12-12 14:22:43,801 - INFO - Shutting down producer...
2024-12-12 14:22:43,801 - INFO - Closing the Kafka producer with 9223372036.0 secs timeout.
2024-12-12 14:22:43,803 - INFO - <BrokerConnection node_id=1002 host-172.25.0.13:9092 <connected> [IPv4 ('172.25.0.13', 9092)]>: Closing connection.
```

Τα μηνύματα που περνάνε στο container του αντίστοιχου kafka topic

```
PS C:\Users\Lampia\Desktop\docker-kafka-spark-master> docker exec -it kafkal kafka-console-consumer.sh --bootstrap-server 172.25.0.13:9092 --topic station_information --from-be-
ginning --max-messages 5
{"external_id": "f8099d58-d26e-468b-9d10-8394372221f", "electric_bike_surcharge_waiver": false, "rental_methods": [{"KEY": "CREDITCARD"}], "lon": -74.013821, "has_kiosk": true,
"eightd_has_key_dispenser": false, "station_type": "classic", "region_id": "71", "rental_uris": [{"ios": "https://bkn.lft.to/lastmile_qr_scan", "android": "https://bkn.lft.to/la
stmile_qr_scan"}], "eightd_station_services": [], "capacity": 22, "short_name": "2012 00", "name": "6 Ave & 69 St", "lat": 40.638190, "station_id": "f8099d58-d26e-468b-9d10-8394
372221f"}
{"external_id": "816e50eb-dc0b-47dc-b773-150e2028b9d", "electric_bike_surcharge_waiver": false, "rental_methods": [{"KEY": "CREDITCARD"}], "lon": -74.009411, "has_kiosk": true,
"eightd_has_key_dispenser": false, "station_type": "classic", "region_id": "71", "rental_uris": [{"ios": "https://bkn.lft.to/lastmile_qr_scan", "android": "https://bkn.lft.to/la
stmile_qr_scan"}], "eightd_station_services": [], "capacity": 19, "short_name": "3084 05", "name": "52 St & 6 Ave", "lat": 40.642703, "station_id": "816e50eb-dc0b-47dc-b773-150e
2028b9d"}
{"external_id": "566ae389-5c22-49ca-8c24-d09ea135170", "electric_bike_surcharge_waiver": false, "rental_methods": [{"KEY": "CREDITCARD"}], "lon": -74.013318, "has_kiosk": true,
"eightd_has_key_dispenser": false, "station_type": "classic", "region_id": "71", "rental_uris": [{"ios": "https://bkn.lft.to/lastmile_qr_scan", "android": "https://bkn.lft.to/la
stmile_qr_scan"}], "eightd_station_services": [], "capacity": 3, "short_name": "3080 03", "name": "55 St & 6 Ave", "lat": 40.642400, "station_id": "566ae389-5c22-49ca-8c24-d09ea
135170"}
{"external_id": "6d76ddaf-9290-4059-a329-1818582d095f", "electric_bike_surcharge_waiver": false, "rental_methods": [{"KEY": "CREDITCARD"}], "lon": -73.90893804061413, "has_kiosk":
true, "eightd_has_key_dispenser": false, "station_type": "classic", "region_id": "71", "rental_uris": [{"ios": "https://bkn.lft.to/lastmile_qr_scan", "android": "https://bkn.l
ft.to/lastmile_qr_scan"}], "eightd_station_services": [], "capacity": 19, "short_name": "4080 01", "name": "Borden Ave & Brevin Ave", "lat": 40.73866335665396, "station_id": "6d
76ddaf-9290-4059-a329-1818582d095f"}
{"external_id": "60de1fe7-0aca-11e7-82f6-30a3b040e76c", "electric_bike_surcharge_waiver": false, "rental_methods": [{"KEY": "CREDITCARD"}], "lon": -73.9014032, "has_kiosk": true,
"eightd_has_key_dispenser": false, "station_type": "classic", "region_id": "71", "rental_uris": [{"ios": "https://bkn.lft.to/lastmile_qr_scan", "android": "https://bkn.lft.to/l
astmile_qr_scan"}], "eightd_station_services": [], "capacity": 27, "short_name": "4060 09", "name": "Carroll St & 5 Ave", "lat": 40.6751622, "station_id": "60de1fe7-0aca-11e7-82
f6-30a3b040e76c"}
Processed a total of 5 messages
PS C:\Users\Lampia\Desktop\docker-kafka-spark-master>
```

Weather information

```
import requests
import json
import logging
import time
from kafka import KafkaProducer

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
BOOTSTRAP_SERVERS = ['172.25.0.13:9092'] # kafka ip:port
WEATHER_TOPIC = "weather_data" # kafka topic name

producer = KafkaProducer( # kafka producer initialization
    bootstrap_servers=BOOTSTRAP_SERVERS,
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)

OPEN_METEO_URL = "https://api.open-meteo.com/v1/forecast" # api for weather info
NEW_YORK_COORDINATES = {"latitude": 40.7128, "longitude": -74.0060} # coordinates of NY

def fetch_weather_data(): # API call
    params = {
        "latitude": NEW_YORK_COORDINATES["latitude"],
        "longitude": NEW_YORK_COORDINATES["longitude"],
        "current_weather": True,
        "timezone": "auto",
        "hourly": "precipitation,cloudcover"
    }
    try:
        response = requests.get(OPEN_METEO_URL, params=params, timeout=10)
        response.raise_for_status()
        weather_data = response.json()
        if "current_weather" in weather_data:
            current_weather = weather_data["current_weather"]
            hourly_data = weather_data.get("hourly", {}) # hourly data contains cloudiness
            precipitation = hourly_data.get("precipitation", [0])[0] if "precipitation" in hourly_data else 0
            cloudiness = hourly_data.get("cloudcover", [None])[0] if "cloudcover" in hourly_data else None
            return {
                "location_name": "New York",
                "latitude": params["latitude"],
                "longitude": params["longitude"],
                "temperature": current_weather.get("temperature"),
                "wind_speed": current_weather.get("windspeed"),
                "precipitation": precipitation,
```

```

        "cloudiness": cloudiness,
        "timestamp": current_weather.get("time"),
    }
else:
    logging.warning("No current weather data found in API response.")
    return None
except requests.RequestException as e:
    logging.error("Error fetching weather data from Open-Meteo: {}".format(e))
    return None

def send_weather_data_for_ny(): # send data to kafka
    weather_data = fetch_weather_data()
    if weather_data:
        try:
            producer.send(WEATHER_TOPIC, value=weather_data)
            logging.info("Sent weather data for New York to Kafka.")
        except Exception as e:
            logging.error("Failed to send weather data to Kafka: {}".format(e))
    else:
        logging.warning("No weather data available for New York.")

if __name__ == "__main__":
    logging.info("Starting Weather Data Producer for New York...")
    try:
        while True:
            send_weather_data_for_ny()
            producer.flush()
            logging.info("Weather data sent successfully! Waiting for the next cycle...")
            time.sleep(100)
    except KeyboardInterrupt:
        logging.info("Shutting down producer...")
        producer.close()

```

Το αποτέλεσμα από την κονσόλα

```

root@62a99575f5c7:/app/producers# python weather_info.py
2024-12-12 14:54:47,468 - INFO - <BrokerConnection node_id=bootstrap-0 host=172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]>: connecting to 172.25.0.13:9092 [('172.25.0.13', 9092) IPv4]
2024-12-12 14:54:47,468 - INFO - Probing node bootstrap-0 broker version
2024-12-12 14:54:47,469 - INFO - <BrokerConnection node_id=bootstrap-0 host=172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]>: Connection complete.
2024-12-12 14:54:47,571 - INFO - Broker version identified as 2.2.0
2024-12-12 14:54:47,571 - INFO - Set configuration api_version=(2, 2, 0) to skip auto check_version requests on startup
2024-12-12 14:54:47,573 - INFO - Starting Weather Data Producer for New York...
2024-12-12 14:54:47,944 - INFO - Sent weather data for New York to Kafka.
2024-12-12 14:54:47,945 - INFO - <BrokerConnection node_id=1002 host=172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]>: connecting to 172.25.0.13:9092 [('172.25.0.13', 9092) IPv4]
2024-12-12 14:54:47,945 - INFO - <BrokerConnection node_id=1002 host=172.25.0.13:9092 <connecting> [IPv4 ('172.25.0.13', 9092)]>: Connection complete.
2024-12-12 14:54:47,945 - INFO - <BrokerConnection node_id=bootstrap-0 host=172.25.0.13:9092 <connected> [IPv4 ('172.25.0.13', 9092)]>: Closing connection.
2024-12-12 14:54:47,948 - INFO - Weather data sent successfully! Waiting for the next cycle...
^C2024-12-12 14:54:48,933 - INFO - Shutting down producer...
2024-12-12 14:54:48,933 - INFO - Closing the Kafka producer with 9223372036.0 secs timeout.
2024-12-12 14:54:48,934 - INFO - <BrokerConnection node_id=1002 host=172.25.0.13:9092 <connected> [IPv4 ('172.25.0.13', 9092)]>: Closing connection.

```


Τα μηνύματα που περνάνε στο container του αντίστοιχου kafka topic

```
Windows PowerShell
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-11T06:45", "longitude": -74.006, "cloudiness": null, "precipitation": 0, "wind_speed": 16.7, "latitude": 40.7128, "temperature": 12.7}
{"location_name": "New York", "timestamp": "2024-12-12T09:45", "longitude": -74.006, "cloudiness": 10, "precipitation": 0.0, "wind_speed": 28.8, "latitude": 40.7128, "temperature": 2.4}
{"location_name": "New York", "timestamp": "2024-12-12T09:45", "longitude": -74.006, "cloudiness": 10, "precipitation": 0.0, "wind_speed": 28.8, "latitude": 40.7128, "temperature": 2.4}
{"location_name": "New York", "timestamp": "2024-12-12T09:45", "longitude": -74.006, "cloudiness": 10, "precipitation": 0.0, "wind_speed": 28.8, "latitude": 40.7128, "temperature": 2.4}
{"location_name": "New York", "timestamp": "2024-12-12T09:45", "longitude": -74.006, "cloudiness": 10, "precipitation": 0.0, "wind_speed": 28.8, "latitude": 40.7128, "temperature": 2.4}
```

Spark Container

Η δημιουργία του container που θα εκτελεστεί το spark γίνεται μέσω του docker compose αρχείου που εκτός από το συγκεκριμένο container δημιουργεί και τα containers για τα kafka/zookeeper. Παρακάτω αποτυπώνεται το συγκεκριμένο αρχείο YAML

```
docker-compose.yml
services:
  zookeeper:
    image: zookeeper:3.6.1
    container_name: zookeeper
    expose:
      - "2181"
    volumes:
      - kafka_zookeeper:/opt/zookeeper-3.6.1/data
    networks:
      kafkanet:
        ipv4_address: 172.25.0.11

  kafka1:
    image: wurstmeister/kafka:2.12-2.2.0
    container_name: kafka1
    command: [start-kafka.sh]
    expose:
      - "8080"
      - "9092"
    environment:
      KAFKA_ADVERTISED_HOST_NAME: 172.25.0.12
      KAFKA_ZOOKEEPER_CONNECT: 172.25.0.11:2181
      KAFKA_ADVERTISED_PORT: 9092
    volumes:
      - kafka_kafka1:/opt/kafka_2.12-2.2.0/logs
    networks:
      kafkanet:
        ipv4_address: 172.25.0.12
    depends_on:
      - "zookeeper"

  kafka2:
    image: wurstmeister/kafka:2.12-2.2.0
    container_name: kafka2
    command: [start-kafka.sh]
    expose:
      - "8080"
      - "9092"
    environment:
      KAFKA_ADVERTISED_HOST_NAME: 172.25.0.13
      KAFKA_ZOOKEEPER_CONNECT: 172.25.0.11:2181
      KAFKA_ADVERTISED_PORT: 9092
    volumes:
      - kafka_kafka2:/opt/kafka_2.12-2.2.0/logs
    depends_on:
      - "zookeeper"
    networks:
      kafkanet:
        ipv4_address: 172.25.0.13

  spark:
    image: gettyimages/spark:2.4.1-hadoop-3.0
    container_name: spark
    volumes:
      - ./:/app
    networks:
      kafkanet:
        ipv4_address: 172.25.0.14

networks:
  kafkanet:
    name: kafkanet
    driver: bridge
    ipam:
      driver: default
```

Όπως αποτυπώνεται και στη γραμμή 53 το container με το spark παίρνει την εικόνα του από την έκδοση 2.4.1. Επιπλέον σε αυτό ανατίθεται και ένας docker volume ώστε τα δεδομένα που παράγονται στο συγκεκριμένο container να μην σβήνονται (persistent data).

Στη συνέχεια αναπτύσσεται ένα πρόγραμμα (bike_sharing_processing.py) με το οποίο γίνονται οι κατάλληλοι μετασχηματισμοί ώστε να φέρουμε τα δεδομένα σε τέτοια μορφή, η οποία θα

δίνει κάποια πληροφορία αλλά και θα αρκεί ώστε να μπορέσουμε να εκπαιδεύσουμε το μοντέλο παλινδρόμησης για την πρόβλεψη της χρήσης των σταθμών.

```
BOOTSTRAP_SERVERS = "172.25.0.13:9092" # Kafka servers
STATION_STATUS_TOPIC = "station_status" # topic 1
WEATHER_TOPIC = "weather_data" # topic 2
STATION_INFO_TOPIC = "station_information" # topic 3

station_status_schema = StructType([ # Schema for station status topic
    StructField("station_id", StringType(), True),
    StructField("num_docks_available", IntegerType(), True),
    StructField("num_ebikes_available", IntegerType(), True),
    StructField("num_bikes_available", IntegerType(), True),
    StructField("num_bikes_disabled", IntegerType(), True),
    StructField("is_renting", IntegerType(), True),
    StructField("num_docks_disabled", IntegerType(), True),
    StructField("eightd_has_available_keys", BooleanType(), True),
    StructField("last_reported", LongType(), True),
    StructField("is_installed", IntegerType(), True),
    StructField("is_returning", IntegerType(), True)
])

weather_schema = StructType([ # Schema for weather info
    StructField("location_name", StringType(), True),
    StructField("timestamp", StringType(), True),
    StructField("longitude", FloatType(), True),
    StructField("latitude", FloatType(), True),
    StructField("cloudiness", IntegerType(), True),
    StructField("precipitation", FloatType(), True),
    StructField("wind_speed", FloatType(), True),
    StructField("temperature", FloatType(), True)
])

station_info_schema = StructType([ # Schema for station info
    StructField("station_id", StringType(), True),
    StructField("name", StringType(), True),
    StructField("lat", FloatType(), True),
    StructField("lon", FloatType(), True),
    StructField("capacity", IntegerType(), True)
])
```

Στο ανωτέρω κομμάτι ορίζονται τα σχήματα για τα δεδομένα ώστε να είναι σε κατάλληλη μορφή για την επεξεργασία.

```

station_status_df = spark.read \
    .format("kafka") \
    .option("kafka.bootstrap.servers", BOOTSTRAP_SERVERS) \
    .option("subscribe", STATION_STATUS_TOPIC) \
    .load()

station_status_parsed = station_status_df.selectExpr("CAST(value AS STRING)").select(
    from_json(col("value"), station_status_schema).alias("parsed")
).select(
    col("parsed.*"),
    from_unixtime(col("parsed.last_reported")).alias("last_updated")
).withColumn(
    "status_hour", hour(col("last_updated"))
).withColumn(
    "status_date", to_date(col("last_updated"))
)

```

Στη συνέχεια αρχικοποιείται η ανάγνωση των δεδομένων και από το topic station status όπου το `format("kafka")` ορίζει ότι η πηγή των δεδομένων θα είναι σε μορφή που στέλνει το kafka, το `option("kafka.bootstrap.servers", BOOTSTRAP_SERVERS)` παρέχει τις οδηγίες στο kafka broker να συνδεθεί και να πάρει τα δεδομένα που έχουν ανακτηθεί από τον producer και το `option("subscribe", STATION_STATUS_TOPIC)` ορίζει από ποιο ακριβώς topic θα διαβαστούν τα μηνύματα. Τέλος η μέθοδος `load()` αρχικοποιεί την διαδικασία φόρτωσης των δεδομένων και ως αποτέλεσμα παράγει ένα dataframe όπου κάθε μήνυμα αποτελεί μία γραμμή.

Συνεχίζοντας η μέθοδος `selectExpr` μετατρέπει κάθε πεδίο από τα μηνύματα σε συμβολοσειρά για περαιτέρω επεξεργασία και το `from_json(col("value"), station_status_schema)` αναλύει τα δεδομένα JSON χρησιμοποιώντας το σχήμα του station status που έχει οριστεί νωρίτερα, μετατρέποντάς τα σε δομημένη μορφή.

Τέλος εισάγει δύο ακόμα στήλες την `status_hour` στην οποία εκχωρεί την ώρα από το πεδίο `last_updated` και την `status_date` που εκχωρεί την ημέρα. Σαν αποτέλεσμα προκύπτει ένα DataFrame όπου κάθε σειρά αντιστοιχεί σε ένα μήνυμα από το station_status μαζί με μια νέα στήλη `last_updated` για ευκολότερη ανάλυση βάσει χρόνου.

Το αποτέλεσμα αποτυπώνεται στην επόμενη σελίδα

Η ίδια διαδικασία ακολουθείται και για τα άλλα δύο topics, δημιουργώντας:

[illegible]

Στη συνέχεια με τη χρήση των παραπάνω views (virtual non-persistent table) παράγεται ένας πίνακας με αναλυτικές πληροφορίες χρήσης και στοιχείων για κάθε σταθμό συνδυασμένες με τα προγνωστικά του καιρού

Joined Data:											
station_id	station_name	latitude	longitude	capacity	available_bikes	docks_available	utilization_rate	last_updated	temperature	wind_speed	cloudiness
66dc4b4c-0aca-11e7-82f6-3863bb44ef7c	W 37 St & 5 Ave	40.75038	-73.98339	60	55	1	0.9821428571428571	2024-12-12 15:37:06	2.8	34.4	10
66dbe571-0aca-11e7-82f6-3863bb44ef7c	W 4 St & 7 Ave S	40.734013	-74.00294	60	12	47	0.2033898305084746	2024-12-12 15:35:42	2.8	34.8	10
66dcca76-0aca-11e7-82f6-3863bb44ef7c	Nostrand Ave & Myrtle Ave	40.69527	-73.95238	23	8	14	0.36363636363636365	2024-12-12 15:34:58	2.8	32.9	10
308ebef6-dcc2-4305-b1d2-47efafdaa85c	Sterling Pl & Rochester Ave	40.67128	-73.92825	22	0	22	0.0	2024-12-12 15:29:34	2.8	34.4	10
e0b80983-c520-47e3-9c8d-a17fff5e5582	E 169 St & Webster Ave	40.834015	-73.98886	21	9	7	0.5625	2024-12-12 15:31:04	2.8	34.8	10
01839a78-f220-4706-87f7-147b057492f2	Adam Clayton Powell Blvd & W 132 St	40.813538	-73.94521	23	4	18	0.18181818181818182	2024-12-12 15:30:28	2.8	34.4	10
66dc702-0aca-11e7-82f6-3863bb44ef7c	Bialystoker Pl & Delancey St	40.716225	-73.98261	33	0	29	0.0	2024-12-12 15:30:10	2.8	34.4	10
341730d7-a61d-499d-8c07-fa015f64ae54	W 37 St & Broadway	40.75227	-73.98771	84	83	0	1.0	2024-12-12 15:30:25	2.8	34.4	10
85ba92ab-8d8a-4d17-9d07-01732d16d3bd	Monroe St & Patchen Ave	40.68814	-73.92693	20	0	18	0.0	2024-12-12 15:30:24	2.8	34.8	10
66dc0e99-0aca-11e7-82f6-3863bb44ef7c	West St & Chambers St	40.71755	-74.01322	115	111	2	0.9823088849557522	2024-12-12 15:30:41	2.8	34.4	10
1d130e43-0408-4423-bb52-77a08372cefa	5 Ave & W 126 St	40.80722	-73.9418	27	16	9	0.64	2024-12-12 15:34:54	2.8	34.8	10
1884688942632618210	29 St & Skillman Ave	40.74483	-73.93841	27	25	0	1.0	2024-12-12 15:34:46	2.8	34.4	10
2a47ef14-1cbe-4ec6-841d-cc277627fe20	Fulton St & Waverly Ave	40.68324	-73.965996	31	9	19	0.32142857142857145	2024-12-12 15:29:21	2.8	32.9	10
66db6d9d-0aca-11e7-82f6-3863bb44ef7c	3 Ave & Schermerhorn St	40.68632	-73.979675	35	11	19	0.36666666666666664	2024-12-12 15:30:45	2.8	34.4	10
1f6cf3ecc-b3ad-4291-8c24-277f28f64ba1	Myckoff Ave & Stanhope St	40.703545	-73.91778	22	18	2	0.9	2024-12-12 15:33:52	2.8	34.4	10
181a00808997407540	Clarkson Ave & E 37 St	40.65577	-73.9432	22	20	0	1.0	2024-12-12 15:34:21	2.8	32.9	10
1aa0e6301-4162-428d-8043-7b604080b0d1	E 157 St & River Ave	40.825727	-73.927315	19	15	3	0.8333333333333334	2024-12-12 15:35:40	2.8	34.8	10
66dc7a7d-0aca-11e7-82f6-3863bb44ef7c	9 Ave & W 22 St	40.7455	-74.00197	61	2	56	0.034482758620689655	2024-12-12 15:35:43	2.8	32.9	10
563380a0-d90f-4155-9c50-528459d4f26aa	Broadway & W 138 St	40.82182	-73.95353	27	8	16	0.3333333333333333	2024-12-12 15:37:01	2.8	34.8	10
66db29e6-0aca-11e7-82f6-3863bb44ef7c	Lexington Ave & Classon Ave	40.686768	-73.95928	19	8	10	0.44444444444444444	2024-12-12 15:34:51	2.7	32.9	10

Διαπιστώνεται ότι

Ο ρυθμός χρήσης ενός σταθμού ποδηλάτων υπολογίζεται βάσει του τύπου:

$$\text{Ρυθμός Χρήσης} = \frac{\text{Διαθέσιμα Ποδήλατα}}{\text{Διαθέσιμα Ποδήλατα} + \text{Διαθέσιμες Θέσεις}}$$

Αυτός ο τύπος εκφράζει την αναλογία των ποδηλάτων που βρίσκονται σε χρήση σε σχέση με τη συνολική δυναμική ενός σταθμού. Για παράδειγμα στον σταθμό με ID: 66dc4b4c-0aca-11e7-82f6-3863bb44ef7c η χωρητικότητα είναι 60 με διαθέσιμα ποδήλατα 55 και διαθέσιμες θέσεις στάθμευσης (dock) μία. Υπολογισμός Ρυθμού Χρήσης: $55 / (55 + 1) = 55 / 56 \approx 0.9821$ (98.21%). Ο σταθμός αυτός παρουσιάζει σχεδόν πλήρη χρήση, καθώς μόνο μία θέση στάθμευσης παραμένει διαθέσιμη.

Αντίθετα στο σταθμό με ID: 66dbe571-0aca-11e7-82f6-3863bb44ef7c υπάρχει χωρητικότητα 60 με διαθέσιμα ποδήλατα 12 και διαθέσιμες θέσεις 47. $(12 / (12 + 47) = 12 / 59 \approx 0.2034)$. Σε αυτόν τον σταθμό, υπάρχει χαμηλότερος ρυθμός χρήσης, καθώς υπάρχουν πολλές διαθέσιμες θέσεις στάθμευσης.

Συνεχίζοντας για την εξαγωγή παραπάνω πληροφορίας και συγκεκριμένα για τη συνολική χρήση του συστήματος αλλά και τη χρήση ανά ώρα έχουν υλοποιηθεί τα παρακάτω sql queries.

```
''' Usage summary per hour and day'''
hourly_usage_summary_query = '''
SELECT
    status_hour, status_date,
    COUNT(DISTINCT station_id) AS active_stations,
    SUM(num_bikes_available) AS total_bikes_available,
    SUM(num_docks_available) AS total_docks_available,
    AVG(temperature) AS avg_temperature,
    AVG(wind_speed) AS avg_wind_speed
FROM
    station_status ss
INNER JOIN
    weather_data wd
ON
    ss.status_date = wd.weather_date
GROUP BY
    status_hour, status_date
ORDER BY
    status_date, status_hour
'''
```

```
system_utilization_query = '''
SELECT
    AVG(utilization_rate) AS avg_utilization_rate,
    MAX(utilization_rate) AS max_utilization_rate,
    MIN(utilization_rate) AS min_utilization_rate,
    AVG(temperature) AS avg_temperature,
    AVG(wind_speed) AS avg_wind_speed,
    AVG(cloudiness) AS avg_cloudiness
FROM
    (
        SELECT
            station_id,
            utilization_rate,
            temperature,
            wind_speed,
            cloudiness,
            precipitation
        FROM
            (

```

```
                SELECT
                    ss.station_id,
                    (ss.num_bikes_available /
                     (ss.num_bikes_available + ss.num_docks_available)) AS utilization_rate,
                    wd.temperature,
                    wd.wind_speed,
                    wd.cloudiness,
                    wd.precipitation
                FROM
                    station_status ss
                INNER JOIN
                    weather_data wd
                ON
                    ss.status_date = wd.weather_date
            ) AS joined_data
        )
    )
```

Και σαν αποτέλεσμα παίρνουμε:

Overall System Utilization:

avg_utilization_rate	max_utilization_rate	min_utilization_rate	avg_temperature	avg_wind_speed	avg_cloudiness
0.5206540835907137	1.0	0.0	2.774999976158142	34.150000381469724	10.0

Hourly Usage Summaries:

status_hour	status_date	active_stations	total_bikes_available	total_docks_available	avg_temperature	avg_wind_speed
10	2024-12-12	1	0	13640	2.774999976158142	34.150000381469724
12	2024-12-12	2	0	7440	2.774999976158142	34.150000381469724
13	2024-12-12	1	0	9920	2.774999976158142	34.150000381469724
15	2024-12-12	2207	43624120	36687400	2.774999976158142	34.150000381469724

Τέλος το παρακάτω κομμάτι κώδικα αποθηκεύει τα δεδομένα σε μία βάση sqlite3 καθώς και σε ένα CSV αρχείο.

```
""" Save to SQLite and CSV """
import sqlite3
import pandas as pd
import os

def save_to_sqlite(df, table_name):
    temp_dir = "/app/temp_output"
    df.coalesce(1).write_csv(temp_dir, header=True, mode="overwrite")

    part_file = [f for f in os.listdir(temp_dir) if f.startswith("part-")][0]
    full_file_path = os.path.join(temp_dir, part_file)
    df_pandas = pd.read_csv(full_file_path)

    conn = sqlite3.connect('/app/bike_sharing.db')
    df_pandas.to_sql(table_name, conn, if_exists="replace", index=False)
    conn.close()
    print("Data saved to SQLite.")

save_to_sqlite(joined_df, "joined_data")
save_to_sqlite(system_utilization_df, "system_utilization")
save_to_sqlite(hourly_usage_summary_df, "hourly_usage_summary")
```


Εκπαίδευση μοντέλου παλινδρόμησης

Στη συνέχεια υλοποιείται ένα ακόμα αρχείο το `bike_processing_plus_ML.py` όπου γίνονται οι ανωτέρω διαδικασίες και στη συνέχεια με τα δεδομένα που έχουν παραχθεί εκπαιδεύεται ένα μοντέλο παλινδρόμησης (random forest) ώστε να προβλέπεται η χρήση ενός σταθμού στην επόμενη ώρα.

Το στιγμιότυπο του σχετικού κώδικα που προσθέτει περαιτέρω χαρακτηριστικά στα δεδομένα για την εκπαίδευση του μοντέλου

```
""" Stats trends for utilization """
utilization_summary = joined_df.groupby("status_date").agg(
    avg("utilization_rate").alias("average_docking_station_utilisation"),
    max("utilization_rate").alias("max_docking_station_utilisation"),
    min("utilization_rate").alias("min_docking_station_utilisation"),
    stddev("utilization_rate").alias("std_dev_docking_station_utilisation")
)

print("Utilization Summary Schema:")
utilization_summary.printSchema()
print("Sample Utilization Summary:")
utilization_summary.show(5, truncate=False)

""" next hour features """
joined_features = joined_df.withColumn("day_of_week", dayofweek(col("last_updated")))
window_spec = Window.partitionBy("station_id").orderBy("last_updated")
joined_features = joined_features.withColumn(
    "next_hour_utilization", lead("utilization_rate", 1).over(window_spec)
).withColumn(
    "forecasted_temperature", lead("temperature", 1).over(window_spec)
).withColumn(
    "forecasted_wind_speed", lead("wind_speed", 1).over(window_spec)
).withColumn(
    "forecasted_cloudiness", lead("cloudiness", 1).over(window_spec)
).filter(col("next_hour_utilization").isNotNull())

ml_features = joined_features.select(
    col("forecasted_temperature").alias("temperature"),
    col("forecasted_wind_speed").alias("wind_speed"),
    col("forecasted_cloudiness").alias("cloudiness"),
    col("hour_of_day"),
    col("capacity"),
    col("next_hour_utilization").alias("utilization_rate"),
```

```

col("station_id"),
col("station_name").alias("name")
)
""" 80% data for training - 20% for testing """
train_data, test_data = ml_features.randomSplit([0.8, 0.2], seed=42)

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import RandomForestRegressor
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml import Pipeline

feature_cols = ["temperature", "wind_speed", "cloudiness", "hour_of_day", "capacity"]
vector_assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
rf_model = RandomForestRegressor(featuresCol="features", labelCol="utilization_rate")

pipeline = Pipeline(stages=[vector_assembler, rf_model])
model = pipeline.fit(train_data)
predictions = model.transform(test_data)
predictions_with_metadata = predictions.select(
    "features", "utilization_rate", "prediction", "station_id", "name"
)

print("Predictions with Metadata Schema:")
predictions_with_metadata.printSchema()
print("Sample Predictions with Metadata:")
predictions_with_metadata.sample(fraction=0.1).distinct().limit(20).show(truncate=False)

""" Model evaluation """
evaluator_rmse = RegressionEvaluator(
    labelCol="utilization_rate", predictionCol="prediction", metricName="rmse"
)

rmse = evaluator_rmse.evaluate(predictions)
print("Root Mean Squared Error (RMSE):", rmse)

""" Model feature importance """
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
features = ["Temperature", "Wind Speed", "Cloudiness", "Hour of Day", "Capacity"]
feature_importances = model.stages[-1].featureImportances.toArray()

plt.figure(figsize=(10, 6))
plt.bar(features, feature_importances, color='orange', alpha=0.7)
plt.title('Feature Contributions to the Model')
plt.xlabel('Features')

```



```
plt.ylabel('Feature Importance')
plt.grid(axis='y')
```

Το group by που γίνεται αρχικά για το utilization summary υπολογίζει σημαντικές μετρήσεις για όλους τους σταθμούς. Πιο συγκεκριμένα υπολογίζει τη χαμηλότερη, μέση και υψηλότερη χρήση ανά σταθμό για κάθε μέρα καθώς και την τυπική απόκλιση για κάθε χρήση ώστε να περιγράψει τη μεταβλητότητα ή την απόκλιση τιμών γύρω από τον μέσο όρο, παρέχοντας μία ανάλυση για το μοντέλο στα μοτίβα χρήσης.

Στη συνέχεια χρησιμοποιώντας ιστορικά δεδομένα μαζί με ένα παράθυρο μίας ώρας, οργανωμένα ανά σταθμό, δημιουργεί μία πληροφορία γύρω από κάθε χρονικό σημείο. Με αυτόν τον τρόπο, το μοντέλο μπορεί να καταλάβει τα μοτίβα που προκύπτουν και πώς αλλάζει το utilization rate και ο καιρός από ώρα σε ώρα. Επίσης πώς οι αλλαγές σε έναν σταθμό, όπως η διαθεσιμότητα ποδηλάτων ή η δυναμικότητα, επηρεάζουν τη χρήση του σταθμού. Συνεπώς το μοντέλο εκπαιδεύεται να προβλέπει πώς οι τρέχουσες συνθήκες επηρεάζουν την κατάσταση στο κοντινό μέλλον. Αυτό το παράθυρο των δεδομένων εξασφαλίζει ότι το μοντέλο λαμβάνει υπόψη τόσο τυχόν παρελθοντική γνώση όσο και τις δυναμικές αλλαγές, επιτρέποντας την καλύτερη κατανόηση και πρόβλεψη. Είναι σημαντικό εδώ να πούμε πώς λόγω ότι ο καιρός δεν παρουσιάζει κάποια μεταβλητότητα καθώς βρισκόμαστε στο χειμώνα και ο καιρός είναι σχετικά ο ίδιος κάθε μέρα αναμένεται πώς το μοντέλο δεν θα είναι σε θέση να κατανοήσει σημαντικές μεταβολές του καιρού που ενδέχεται να παρουσιάζουν αλλαγή στη χρήση των σταθμών παρά μόνο μία σχετικά μικρή επιρροή όσον αφορά την θερμοκρασία.

Με την μεταβλητή `ml_features` επιλέγονται τα σχετικά δεδομένα που θα εκπαιδευτεί το μοντέλο. Συγκεκριμένα, θερμοκρασία, ταχύτητα του ανέμου και επίπεδο συννεφιάς, χωρητικότητα και χρήση των σταθμών. Τα δεδομένα διαχωρίζονται σε 2 σετ (εκπαίδευσης και επικύρωσης – 90%/10%).

Ο αλγόριθμος παλινδρόμησης Random Forest αποτελεί μία μέθοδο εκμάθησης που συνδυάζει πολλαπλά δέντρα απόφασης για την παραγωγή προβλέψεων και μια τεχνική που ονομάζεται bagging. Η βασική ιδέα πίσω από αυτό είναι ο συνδυασμός πολλαπλών δέντρων αποφάσεων για τον προσδιορισμό της τελικής πρόβλεψης αντί να βασίζεται σε μεμονωμένα δέντρα απόφασης

Στη συνέχεια το κομμάτι κώδικα παρακάτω συγκεντρώνει τα χαρακτηριστικά εκπαίδευσης σε ένα διάνυσμα χρησιμοποιώντας την κλάση VectorAssembler από τη βιβλιοθήκη pyspark και χρησιμοποιεί το `RandomForestRegressor` για την πρόβλεψη του utilization rate.

```
vector_assembler = VectorAssembler(inputCols=feature_cols, outputCol="features")
rf_model = RandomForestRegressor(featuresCol="features", labelCol="utilization_rate")

pipeline = Pipeline(stages=(vector_assembler, rf_model))
```

Αποτελέσματα

Το Schema παρέχει μια δομημένη αναπαράσταση για τη χρήση των σταθμών. Πιο συγκεκριμένα το status_date αντιπροσωπεύει την ημερομηνία για την οποία υπολογίζονται οι προβλέψεις. Το average_docking_station_utilisation εμφανίζει το μέσο ποσοστό χρήσης των σταθμών για μια συγκεκριμένη ημέρα. Το max_docking_station_utilisation δείχνει το μέγιστο ποσοστό χρήσης που παρατηρήθηκε σε οποιονδήποτε σταθμό για μια συγκεκριμένη ημέρα. Το min_docking_station_utilisation υποδεικνύει το ελάχιστο ποσοστό χρήσης σε οποιονδήποτε σταθμό την ίδια ημέρα. Τέλος το std_dev_docking_station_utilisation αντανακλά τη μεταβλητότητα στα ποσοστά χρήσης μεταξύ όλων των σταθμών.

```
### Spark Session Initialized ###
Utilization Summary Schema:
root
|-- status_date: date (nullable = true)
|-- average_docking_station_utilisation: double (nullable = true)
|-- max_docking_station_utilisation: double (nullable = true)
|-- min_docking_station_utilisation: double (nullable = true)
|-- std_dev_docking_station_utilisation: double (nullable = true)
```

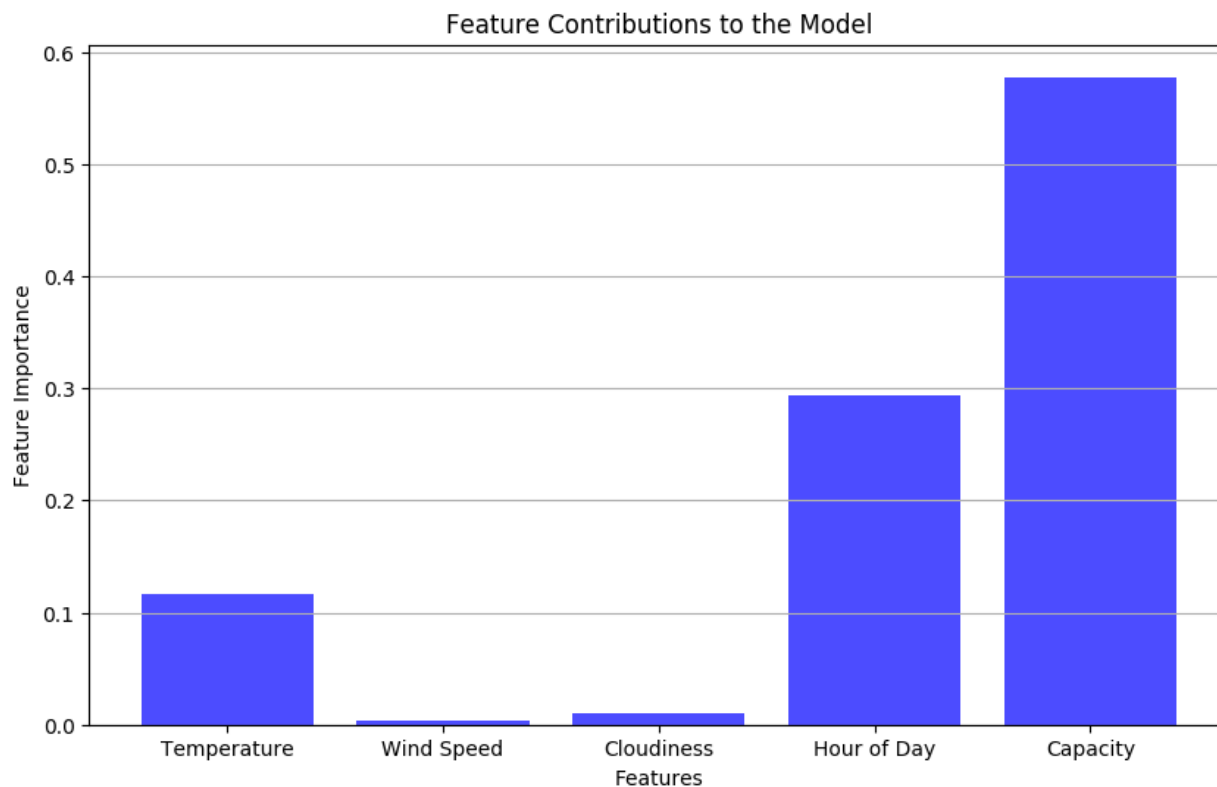
```
Sample Predictions with Metadata:
+-----+-----+-----+-----+-----+
|features|utilization_rate|prediction|station_id|name|
+-----+-----+-----+-----+-----+
|[-1.5,6.199999889265137,100.0,22.0,22.0]|0.8571428571428571|0.5122492169415863|8f72fe26-98aa-4e87-a612-9ae0eb1d9a38|62 St & 34 Ave|
|[-1.5,6.199999889265137,100.0,22.0,26.0]|0.96|0.5122492169415863|1814907423052790164|E 16 St & Cortelyou Rd|
|[-1.5,6.199999889265137,100.0,22.0,33.0]|0.7272727272727273|0.5205977940056283|548cb08c-4ec1-46d9-8978-1a8d3bf42454|34 Ave & 38 St|
|[-1.5,6.199999889265137,100.0,22.0,34.0]|1.0|0.551787847162909|66dd26f9-0aca-11e7-82f6-3863bb44ef7c|9 St & 44 Rd|
|[-1.5,6.199999889265137,100.0,22.0,47.0]|0.13636363636363635|0.5712228770335661|66de88d7-0aca-11e7-82f6-3863bb44ef7c|Fulton St & Adams St|
|[2.774999976158142,34.150000381469724,10.0,15.0,22.0]|0.8888888888888888|0.5055681382977845|8f72fe26-98aa-4e87-a612-9ae0eb1d9a38|62 St & 34 Ave|
|[2.774999976158142,34.150000381469724,10.0,15.0,26.0]|0.9166666666666666|0.5055681382977845|1814907423052790164|E 16 St & Cortelyou Rd|
|[2.774999976158142,34.150000381469724,10.0,15.0,33.0]|0.7272727272727273|0.510067889466573|548cb08c-4ec1-46d9-8978-1a8d3bf42454|34 Ave & 38 St|
|[2.774999976158142,34.150000381469724,10.0,15.0,34.0]|1.0|0.510067889466573|66dd26f9-0aca-11e7-82f6-3863bb44ef7c|9 St & 44 Rd|
|[2.774999976158142,34.150000381469724,10.0,15.0,47.0]|0.6923076923076923|0.5816621637746534|66de88d7-0aca-11e7-82f6-3863bb44ef7c|Fulton St & Adams St|
|[-1.5,6.199999889265137,100.0,22.0,14.0]|0.14285714285714285|0.5165088614253242|66dd9183-0aca-11e7-82f6-3863bb44ef7c|Baldwin at Montgomery|
|[-1.5,6.199999889265137,100.0,22.0,17.0]|0.13333333333333333|0.5165088614253242|837b7db3-55c6-4595-a497-834972631752|Stanhope St & Fairview Ave|
|[-1.5,6.199999889265137,100.0,22.0,19.0]|0.2631578947368421|0.5179795370753075|1057229c-f8f8-4014-80cd-f40421c61cf0|Onderdonk Ave & Gates Ave|
|[-1.5,6.199999889265137,100.0,22.0,19.0]|0.9473684210526315|0.5179795370753075|956de8aa-9e06-4efb-8eb0-35fe9b274a48|60 St & 60 Rd|
|[-1.5,6.199999889265137,100.0,22.0,23.0]|0.0|0.5122492169415863|ef6216aa-d3c8-46bb-8c6b-b62f8ba51b16|Van Sinderen Ave & Truxton St|
|[-1.5,6.199999889265137,100.0,22.0,24.0]|0.4090909090909091|0.5122492169415863|1975559886658995034|5 Ave & 44 St|
|[-1.5,6.199999889265137,100.0,22.0,24.0]|0.5|0.5122492169415863|66bd2d2b-0aca-11e7-82f6-3863bb44ef7c|Clinton St & Joralemon St|
|[-1.5,6.199999889265137,100.0,22.0,28.0]|1.0|0.5122492169415863|66de0d91-0aca-11e7-82f6-3863bb44ef7c|Union St & 4 Ave|
|[-1.5,6.199999889265137,100.0,22.0,29.0]|0.0|0.5122492169415863|7dccc5a03-5997-4250-9207-fa8b6f027c86|Somers St & Broadway|
|[-1.5,6.199999889265137,100.0,22.0,41.0]|0.325|0.5342233047398571|66dd4d70-0aca-11e7-82f6-3863bb44ef7c|Riverside Dr & W 72 St|
+-----+-----+-----+-----+-----+

Root Mean Squared Error (RMSE): 0.3400629376348384
Feature importance plot saved to: model_rmse_feature_importance.png
Mean Absolute Error (MAE): 0.30207329788498627
Metadata saved...
Predictions saved...
Feature importances saved!
```

Τα αποτελέσματα δείχνουν το utilization rate ενός σταθμού καθώς και την πρόβλεψη για την επόμενη ώρα και τα στοιχεία των σταθμών (station id, όνομα).

Μετρώντας την απόδοση του μοντέλου το μέσο τετραγωνικό σφάλμα (RMSE): 0.34, υποδηλώνει ότι κατά μέσο όρο οι προβλέψεις αποκλίνουν από τις πραγματικές τιμές κατά περίπου 34%. Τέλος το Μέσο Απόλυτο Σφάλμα (MAE): 0.3, αντιπροσωπεύει τη μέση απόλυτη διαφορά μεταξύ της πραγματικής και της προβλεπόμενης τιμής. Τέλος στην επόμενη γραφική

απεικονίζεται πόσο επηρεάζει το κάθε χαρακτηριστικό στην πρόβλεψη του μοντέλου με τη χωρητικότητα του εκάστοτε σταθμού να είναι το πιο βασικό ακολουθούμενο με την ώρα της ημέρας και τη θερμοκρασία.

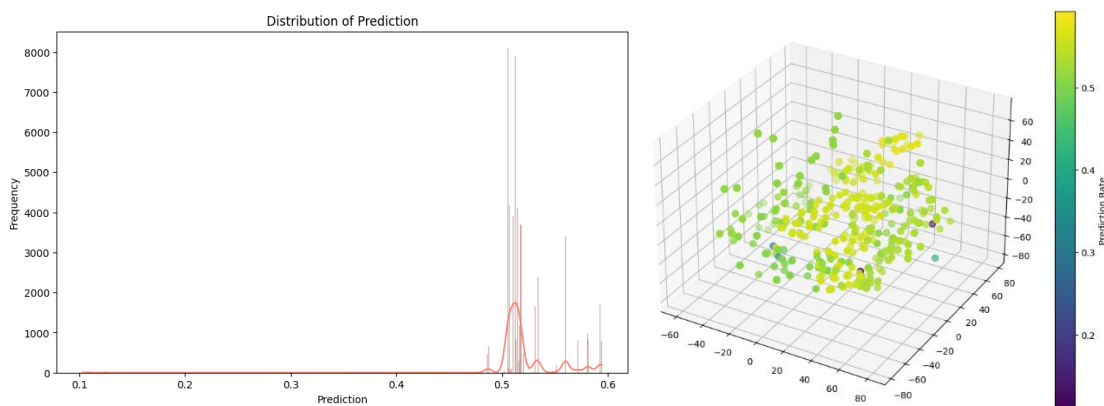


Εικόνα 1 Επιρροή χαρακτηριστικών στις προβλέψεις

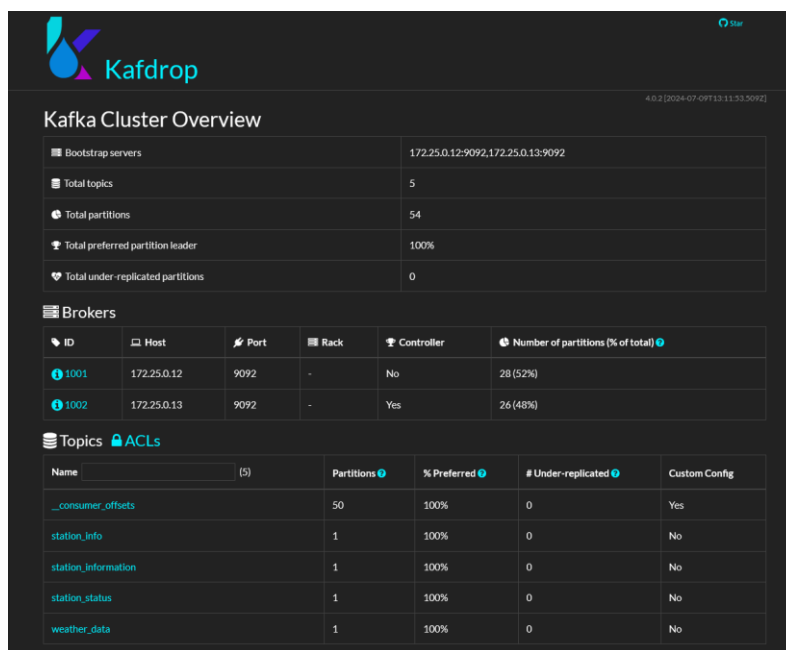
Συμπεράσματα

Λόγω της φύσης των δεδομένων χρονοσειράς αλλά και των πολύπλοκων μοτίβων που υπόκεινται στα δεδομένα των σταθμών ο Random Forest δεν είχε και πολύ ακριβή αποτελέσματα καθώς πρόκειται για ένα γραμμικό μοντέλο. Επιπλέον από τη γραφική παράσταση της εικόνας 2 παρατηρείται ότι το μοντέλο τείνει να προβλέπει χρήση σταθμών κοντά στο 0.5. Στο σημείο αυτό αξίζει να αναφερθεί ότι το μοντέλο παρουσίασε παρόμοια αποτελέσματα με μία πιο βαθεία αρχιτεκτονική (περισσότερα και πιο βαθεία δέντρα απόφασης) αλλά και κάνοντας χρήση περισσότερων χαρακτηριστικών. Στα μη γραμμικά μοτίβα που επικρατούν, καθίσταται δύσκολη η σύγκλιση του, ιδίως όταν τα δεδομένα παρουσιάζουν πολυπλοκότητα. Για το λόγο αυτό η διαδικασία πρόβλεψης θα επωφελούνταν σημαντικά εάν εφαρμόζονταν πιο εξειδικευμένα μοντέλα όπως LSTMs, autoencoders ή και transformers. Πιο συγκεκριμένα, τα LSTMs έχουν σχεδιαστεί για να χειρίζονται δεδομένα χρονοσειρών και είναι ιδιαίτερα αποτελεσματικά όταν τα προηγούμενα χρονικά βήματα επηρεάζουν σημαντικά το παρόν. Εάν το ποσοστό χρήσης εξαρτάται από χρονικά μοτίβα, ένα LSTM μπορεί να μάθει

αυτές τις εξαρτήσεις. Επιπλέον θα μπορούσε να χρησιμοποιηθεί ένας autoencoder για τη συμπίεση των δεδομένων και την εισαγωγή τους στο Random Forest καθώς οι autoencoders μπορούν να συλλαμβάνουν πολύπλοκα μοτίβα σε υψηλών διαστάσεων δεδομένα, μειώνοντας τα σε μία συμπίεσμένη αναπαράσταση και στη συνέχεια ανακατασκευάζοντας τα. Τέλος, εάν και υπολογιστικά ασύμφοροι, οι transformers θα μπορούσαν να βοηθήσουν καθώς είναι ισχυροί στην μοντελοποίηση εξαρτήσεων στα δεδομένα. Σε αντίθεση με τα LSTMs, τα οποία επεξεργάζονται διαδοχικά δεδομένα βήμα προς βήμα, οι transformers μπορούν να παρακολουθήσουν όλα τα μέρη της ακολουθίας ταυτόχρονα χρησιμοποιώντας το attention mechanism.



Εικόνα 2 Κατανομή των προβλέψεων



Εικόνα 3 Απεικόνιση των topics από το kafdrop dashboard